

Riorganizzazione della Blood Supply Chain in Campania: Modello MILP e Strategie Euristiche

Abbagnano Alessia, Astarita Vincenzo, Coppola Alessandro

June 16, 2026

Abstract

La gestione della *Blood Supply Chain* (BSC) è fondamentale per garantire disponibilità, efficienza e sicurezza del sangue destinato a trasfusioni. Questo lavoro descrive un progetto di riorganizzazione della rete regionale dei centri trasfusionali in Campania, partendo da linee guida nazionali e vincoli di produttività minima. Come modello base, è stato utilizzato quello proposto nel paper *A MILP Formulation for the Reorganization of the Blood Supply Chain in Italian Regions* di Antonio Diglio, Andrea Mancuso, Adriano Masone, Carmela Piccolo e Claudio Sterle. Su questa base, abbiamo implementato un modello di programmazione lineare intera mista (MILP) per ottimizzare la posizione e il numero di centri, con una funzione obiettivo che minimizza i costi logistici e le penalità per inefficienze. Per ottenere risultati rapidi e scalabili, sono state inoltre sviluppate euristiche di tipo *Greedy*, tra cui una versione “al contrario”. La relazione documenta tutto il processo, dalla modellazione alla valutazione dei risultati, evidenziando come le euristiche siano state applicate per produrre soluzioni efficienti a partire dal modello originale del paper.

1 Contesto Sanitario e Problema

Il sangue è una risorsa salvavita senza sostituti artificiali. La BSC italiana è interamente basata su donazioni volontarie e non retribuite, gestite dal Servizio Sanitario Nazionale. La fragilità della catena deriva dalla limitata disponibilità di donatori, dall’elevata deperibilità del sangue e dalla necessità di coordinamento di raccolta, test, processamento e distribuzione.

Nel 2012, in applicazione delle direttive europee, il Ministero della Salute ha imposto che ogni Blood Center (BC) raggiunga una produttività minima di 40.000 unità annue. In Campania, la media attuale è di 6.890 unità per BC, rendendo necessaria una riorganizzazione strutturale per garantire autosufficienza regionale e standard di efficienza.

2 Modello di Ottimizzazione MILP

2.1 Scelte strategiche e definizione del problema

La riorganizzazione della Blood Supply Chain (BSC) regionale implica due decisioni strategiche principali:

1. Quali strutture mantenere operative, declassare o chiudere.
2. Come assegnare ogni donatore ai centri disponibili, includendo Blood Centers (BC), Blood Stations (BS) e unità mobili.

Ogni struttura può essere:

- Aperta come BC: centro completo con raccolta e processamento.
- Declassata a BS: mantiene solo la raccolta e invia il sangue a un BC.
- Chiusa: non più utilizzata.

Il problema è modellato su uno spazio di localizzazione corrispondente a una regione con una domanda annua di sangue D .

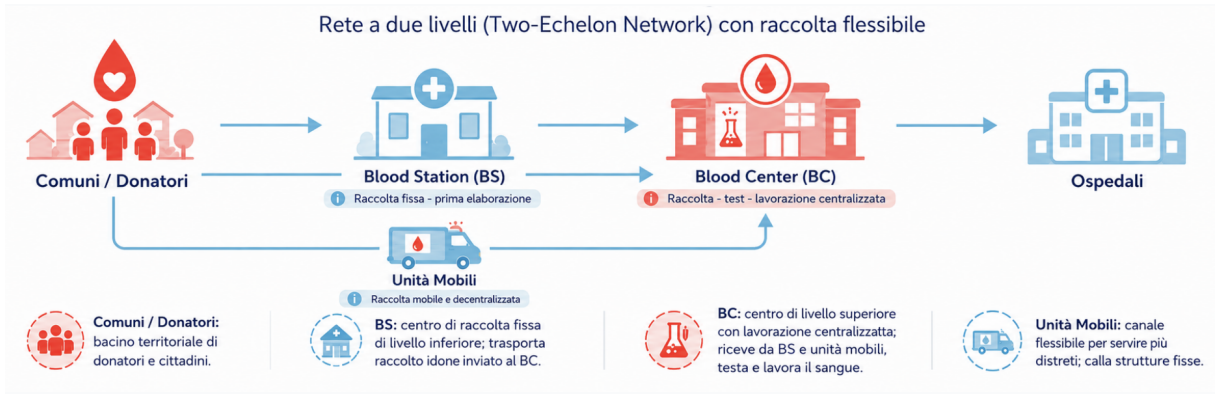


Figure 1: Struttura della Blood Supply Chain

3 Funzione Obiettivo e Variabili del Modello

Il cuore del modello MILP sviluppato per la riorganizzazione del Blood Supply Chain regionale è rappresentato dalla funzione obiettivo, che mira a minimizzare i costi complessivi del sistema, considerando diversi aspetti operativi e penalità.

3.1 Variabili del Modello

Le variabili del modello si suddividono in tre categorie principali:

- **Variabili di Nodo (binaria):**

- y_j^s : vale 1 se nel nodo j è attiva una Blood Station (BS), 0 altrimenti.
- y_j^c : vale 1 se nel nodo j è attivo un Blood Center (BC), 0 altrimenti.

- **Variabili di Assegnazione (binaria):**

- x_{ij} : vale 1 se tutti i potenziali donatori del nodo i sono assegnati alla struttura j .
- $q_{jj'}$: vale 1 se il sangue raccolto dalla struttura j è processato dalla struttura j' .

- z_{ij} : vale 1 se i donatori del nodo i sono assegnati al BC j tramite unità mobili.
- $t_{ijj'}$: vale 1 se i donatori del nodo i , inizialmente assegnati a j , sono riallocati a j' .
- $w_{ij'j}$: vale 1 se $x_{ij'}$ e $q_{j'j}$ sono entrambe 1, utilizzata per linearizzare il prodotto $x_{ij'} \cdot q_{j'j}$.

- **Variabili di Penalità (continua, non negativa):**

- ϕ_j : carenza di sangue processato rispetto al target minimo $P_{\min}$ per il BC j .
- ψ_j : surplus rispetto alla capacità massima C della struttura j .
- δ : scarsità rispetto al target regionale di autosufficienza D .

3.2 Funzione Obiettivo

La funzione obiettivo considera quattro componenti principali di costo:

$$\min Z = \underbrace{\sum_{i \in I} \sum_{j \in J} \sum_{j' \in J} a_i d_{jj'} w_{ij'j}}_{\text{Trasporto BS} \rightarrow \text{BC}} + \underbrace{\sum_{i \in I} \sum_{j \in J} a_i d_{ij} z_{ij}}_{\text{Raccolta tramite unità mobili}} + \underbrace{\sum_{i \in I} \sum_{j \in J} \sum_{j' \in J} a_i d_{jj'} t_{ijj'}}_{\text{Riallocazione dei donatori}} + \underbrace{\sum_{j \in J} \lambda(\phi_j + \psi_j + \delta)}_{\text{Penalità per vincoli non rispettati}} \quad (1)$$

- **Trasporto BS $\rightarrow$ BC:** i costi legati al trasferimento del sangue dalle Blood Station ai Blood Center.
- **Raccolta tramite unità mobili:** il costo della raccolta diretta nel nodo dei donatori tramite unità mobili.
- **Riallocazione dei donatori:** costo aggiuntivo per trasferire i donatori da un BC a un altro in caso di eccedenza.
- **Penalità:** costi associati al mancato rispetto dei vincoli di autosufficienza regionale, produttività minima dei BC o capacità massima delle strutture.

Questa formulazione consente al solutore esatto (Gurobi) di trovare la combinazione ottimale di strutture da mantenere aperte, da declassare e da chiudere, minimizzando i costi di trasporto e le penalità legate al mancato rispetto dei vincoli di gestione.

3.3 Vincoli del Modello MILP

Nel modello MILP sviluppato per la riorganizzazione della Blood Supply Chain (BSC), i vincoli fondamentali possono essere suddivisi in tre gruppi principali: localizzazione e assegnazione, instradamento verso i Blood Center e gestione di unità mobili e riallocazioni. Di seguito sono riportati tutti i vincoli implementati, numerati come nel paper originale.

3.4 Localizzazione e Assegnazione

$$y_j^{(s)} + y_j^{(c)} \leq 1 \quad \forall j \in J \quad (2)$$

$$x_{ij} \leq y_j^{(s)} + y_j^{(c)} \quad \forall i \in I, \forall j \in N_i \quad (3)$$

$$x_{ij} = 0 \quad \forall i \in I, \forall j \notin N_i \quad (4)$$

$$|N_i| \sum_{j \in N_i} x_{ij} \geq \sum_{j \in N_i} (y_j^{(s)} + y_j^{(c)}) \quad \forall i \in I \quad (5)$$

$$\sum_{j \in N_i} d_{ij} x_{ij} + (F - d_{ij})(y_j^{(s)} + y_j^{(c)}) \leq F \quad \forall i \in I, \forall j \in N_i \quad (6)$$

3.5 Instradamento verso i Blood Center

$$q_{jj'} \leq y_{j'}^{(c)} \quad \forall j, j' \in J \quad (7)$$

$$\sum_{j' \in M_j} q_{jj'} = y_j^{(s)} + y_j^{(c)} \quad \forall j \in J \quad (8)$$

$$\sum_{j' \notin M_j} q_{jj'} = 0 \quad \forall j \in J \quad (9)$$

$$q_{jj} \geq y_j^{(c)} \quad \forall j \in J \quad (10)$$

3.6 Unità mobili e riallocazioni

$$\sum_{j \in J} (x_{ij} + z_{ij}) \leq 1 \quad \forall i \in I \quad (11)$$

$$z_{ij} \leq y_j^{(c)} \quad \forall i \in I, \forall j \in O_i \quad (12)$$

$$\sum_{j \notin O_i} z_{ij} = 0 \quad \forall i \in I \quad (13)$$

$$\sum_{j \in J} z_{ij} \leq 1 \quad \forall i \in I \quad (14)$$

$$t_{ijj'} \leq x_{ij} \quad \forall i \in I, \forall j, j' \in J \quad (15)$$

$$t_{ijj'} \leq y_{j'}^{(c)} \quad \forall i \in I, \forall j, j' \in J \quad (16)$$

3.7 Domanda, capacità e linearizzazione

$$\sum_{i \in I} \sum_{j \in J} a_i x_{ij} + \sum_{i \in I} \sum_{j \in J} a_i z_{ij} + \delta \geq D \quad (17)$$

$$\sum_{i \in I} a_i x_{ij} - \sum_{i \in I} \sum_{j' \in J} a_i t_{ijj'} - \psi_j \leq C \quad \forall j \in J \quad (18)$$

$$\sum_{i \in I} \sum_{j' \in J} a_i (w_{ij'j} - t_{ij'j}) - \sum_{i \in I} \sum_{j' \neq j} a_i t_{ijj'} + \sum_{i \in I} a_i z_{ij} + \phi_j \geq P_{\min} y_j^{(c)} \quad \forall j \in J \quad (19)$$

$$\sum_{i \in I} \sum_{j \in J} a_i z_{ij} + \sum_{i \in I} \sum_{j \in J} \sum_{j' \in J} a_i t_{ijj'} \leq t_n \quad (20)$$

$$w_{ijj'} \leq x_{ij} \quad \forall i \in I, \forall j, j' \in J \quad (21)$$

$$w_{ijj'} \leq q_{jj'} \quad \forall i \in I, \forall j, j' \in J \quad (22)$$

$$w_{ijj'} \geq x_{ij} + q_{jj'} - 1 \quad \forall i \in I, \forall j, j' \in J \quad (23)$$

$$y_j^{(s)}, y_j^{(c)}, x_{ij}, q_{jj'}, w_{ijj'}, z_{ij}, t_{ijj'} \in \{0, 1\} \quad \forall i \in I, \forall j, j' \in J \quad (24)$$

$$\phi_j, \psi_j \geq 0, \quad \delta \geq 0 \quad \forall j \in J \quad (25)$$

Questa formulazione mostra chiaramente tutte le regole implementate nel modello MILP: la prima parte definisce la scelta del tipo di struttura e l'assegnazione dei donatori; la seconda parte regola l'instradamento del sangue verso i BC; la terza parte controlla le unità mobili e le riallocazioni; infine vengono imposti i vincoli di domanda, capacità massima e le variabili di penalità.

Questa formulazione permette di simulare scenari realistici di riorganizzazione della rete, bilanciando l'obiettivo di autosufficienza con l'efficienza dei costi logistici, e fornisce una base solida per l'ottimizzazione esatta con solutori MILP come Gurobi.

4 Generazione dell'istanza e costruzione del grafo di rete

Per simulare il problema di riorganizzazione della Blood Supply Chain in Campania, l'istanza è stata costruita seguendo quattro passaggi principali.

4.1 Nodi di domanda I

L'insieme dei nodi di domanda comprende tutti i 599 comuni della regione Campania. La mappa della regione è stata scaricata tramite **OpenStreetMap** e, per ogni comune, è stato calcolato il centroide geografico utilizzando Python. Questo centroide rappresenta il nodo del comune all'interno della rete. La città di Napoli, per garantire maggiore dettaglio, è stata suddivisa in 10 municipalità, così come indicato nel paper di riferimento.

4.2 Popolazione e donatori attivi

Per tutti i comuni, eccetto Napoli, la popolazione è stata generata casualmente nel range $[2000, 50000]$. Per Napoli, la popolazione complessiva è stata ripartita tra le 10 municipalità. Il numero di donatori attivi in ciascun nodo è stato calcolato applicando il tasso di donazione $\alpha = 0.035$ alla popolazione del nodo, come suggerito dal paper.

4.3 Nodi candidati J

Tra i nodi territoriali sono stati selezionati casualmente 22 centri candidati. Ogni centro candidato può essere scelto dal modello come Blood Center (BC), Blood Station (BS) oppure può rimanere chiuso, a discrezione della soluzione ottimale del modello MILP.

4.4 Costruzione degli archi e delle distanze

Gli archi del grafo collegano i comuni ai centri candidati e i centri tra di loro. I pesi degli archi sono stati calcolati a partire dalle coordinate di latitudine e longitudine, utilizzando la distanza euclidea convertita in chilometri, secondo la formula:

$$d = \sqrt{(\Delta\text{lat})^2 + (\Delta\text{lon})^2} \cdot 111.3$$

In questo modo, è stata costruita una rappresentazione completa della rete territoriale, utile per calcolare le distanze di viaggio dei donatori verso le strutture e tra le strutture per l'instradamento del sangue raccolto.

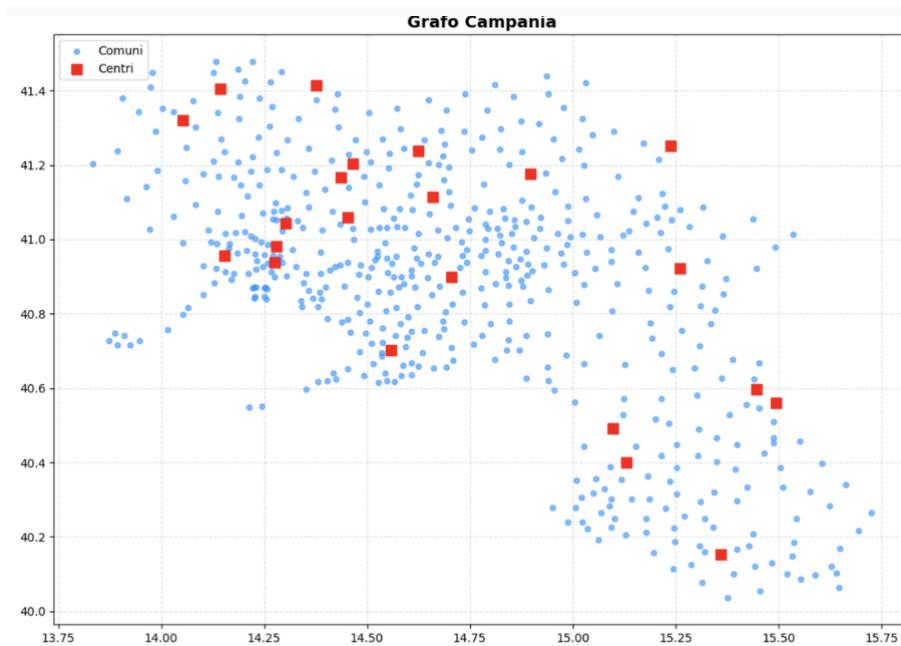


Figure 2: Grafo Campania con comuni e centri

4.5 Discussione

Questo modello permette di trovare la combinazione ottimale di BC e BS aperti, considerando vincoli logistici e produttivi. È stato implementato in Python usando Gurobi,

con tutti gli insiemi, parametri e penalità definiti come nel paper originale. La modellazione consente di simulare anche scenari “what-if” variando la domanda, capacità, tasso di donazione e raggio massimo.

5 Implementazione Python con Gurobi

Al fine di valutare la bontà della modellazione MILP proposta, il modello è stato implementato in Python su **Gurobi** e testato per la regione Campania. Sono stati definiti i parametri principali come segue: il target di autosufficienza regionale $D = 500,000$ unità, la capacità massima di raccolta di ogni struttura $C = 30,000$, la produttività minima obbligatoria di un Blood Center $P_{min} = 40,000$ unità, e il budget massimo disponibile per le unità mobili $tn = 100,000$ unità.

Sono stati utilizzati i dati geografici dei comuni (OSMnx) e generati centri casuali. La costruzione degli insiemi N_i, M_j, O_i e dei parametri a_i è stata automatizzata.

```
# Esempio: creazione insiemi N, O, M
N = {i: [j for j in J if dist_comuni_centri[(i,j)] <= raggio_donatori] for
O = {i: [j for j in J if dist_comuni_centri[(i,j)] <= raggio_inter_centro]
M = {j: [j_prime for j_prime in J if j == j_prime or dist_centri_centri[(j,
# Offerta potenziale
a = {i: comuni[i][2] * 0.035 for i in I}
```

Tutti i vincoli MILP sono stati implementati rispettando le specifiche del paper, comprese le riassegnazioni e penalità.

5.1 Risultati

I risultati ottenuti mostrano la seguente distribuzione finale delle strutture:

- **Blood Centers (BC) aperti:** 11
- **Blood Stations (BS) aperte:** 9

Il valore della funzione obiettivo risultante è stato $Z = 3.261 \cdot 10^9$, con un tempo di esecuzione di circa 15.8 secondi. Questi dati rappresentano il benchmark ottimale ottenuto tramite il solver esatto e serviranno come riferimento per confrontare le performance delle euristiche sviluppate, quali l'algoritmo *Greedy* e il *Greedy al contrario*, valutando l'efficienza delle soluzioni approssimate rispetto all'ottimo.

L'analisi di questa soluzione fornisce indicazioni strategiche sulla scelta dei centri attivi e sulla distribuzione dei flussi di sangue nella rete, evidenziando come sia possibile ridurre il numero di centri mantenendo un livello di autosufficienza e produttività conforme ai vincoli del modello.

6 Euristiche Greedy

Per affrontare il problema della riorganizzazione della rete di raccolta e processamento del sangue in Campania, sono state implementate due euristiche di tipo Greedy, utilizzando la funzione di valutazione sviluppata in Python per stimare la qualità di ogni configurazione candidata.

Le due euristiche considerate sono:

- **Greedy Classico:** un approccio costruttivo che seleziona iterativamente i centri da aprire come Blood Center (BC) o Blood Station (BS) sulla base della minimizzazione della funzione obiettivo, considerando la logistica, le capacità di raccolta e le penalità associate a vincoli non rispettati.
- **Drop Greedy (Greedy al Contrario):** partendo da una configurazione iniziale con tutti i centri attivi come BC, la procedura valuta per ogni centro se conviene degradarlo a BS o chiuderlo, scegliendo in ciascuna iterazione l'opzione che riduce maggiormente la funzione obiettivo.

6.1 Funzione di valutazione

Per entrambi l'euristiche è stata costruita una funzione `valutazione(bc_aperte, bs_aperte)` per calcolare il valore della funzione obiettivo Z associato a una configurazione della rete, ossia dato un insieme di Blood Center (BC) aperti e di Blood Station (BS) aperte.

Gestione comuni: Per ogni comune i , la funzione cerca prima le strutture attive raggiungibili entro il raggio di copertura dei donatori. Se esistono strutture vicine, il comune viene assegnato alla struttura più vicina e il relativo volume di sangue a_i viene aggiunto alla raccolta della struttura.

Gestione unità mobili: Se invece nessuna struttura è vicina, il comune viene servito tramite unità mobili, assegnandolo al BC più vicino entro il raggio consentito, rispettando il limite massimo dell'unità mobile tn . In questo caso viene aggiunto anche il costo logistico proporzionale alla distanza. Se invece si è già sfornato tn , il comune resta isolato, ovvero non servito.

Gestione eccedenze: Successivamente la funzione controlla eventuali superamenti della capacità massima C . Se una struttura raccoglie più sangue del limite massimo, alcuni comuni assegnati vengono riallocati verso il BC più vicino disponibile, generando un costo aggiuntivo.

Gestione trasferimento BS→BC: Infine, la funzione instrada il sangue raccolto dalle BS verso i BC, assicurando che il flusso sia compatibile con le distanze massime consentite tra strutture e rispettando la capacità delle unità mobili.

Gestione sangue BC: Si simula l'attività di processamento del sangue raccolto dalle singole BC nelle stesse BC ai fini del calcolo della funzione obiettivo Z .

Calcolo funzione obiettivo Vengono calcolate le penalità per:

- carenze rispetto al target di autosufficienza regionale D (`delta`),
- mancanze rispetto alla produttività minima dei BC $P_{\min}$ (`somma_phi`),
- superamenti della capacità massima C dei centri (`somma_psi`).

La funzione obiettivo Z viene quindi calcolata come somma del costo logistico più le penalità, ossia:

$$Z = \text{costo_logistico} + \lambda \cdot (\delta + \sum \phi_j + \sum \psi_j)$$

dove λ rappresenta il peso della penale nella funzione obiettivo.

Calcolo delle penalità e della funzione obiettivo

`delta = max(0.0, D - sangue_totale_raccolto)`

`somma_phi = sum(max(0.0, P_min - processato_bc[bc])
for bc in bc_aperti)`

`somma_psi = sum(max(0.0, raccolta_struttura[j] - C)
for j in (bc_aperti + bs_aperte))`

Funzione obiettivo totale

`Z = costo_logistico + lambda_penalita * (delta + somma_phi + somma_psi)`

6.2 Greedy Classica

L'algoritmo parte da una rete vuota, quindi senza BC e BS aperte, e inizializza il costo corrente a infinito. Ad ogni iterazione vengono analizzati solo i centri ancora chiusi. Per ogni candidato j , il codice valuta due possibili aperture: prima come BC, poi come BS, ma la BS può essere aperta solo se esiste già almeno un BC attivo.

La logica Greedy consiste nello scegliere a ogni passo il nodo e il ruolo che producono il valore più basso della funzione obiettivo Z . Se nessuna scelta migliora la soluzione corrente, l'algoritmo termina; altrimenti aggiorna la rete aggiungendo il centro scelto alla lista dei BC o delle BS aperte.

```
def Greedy():
    bc_aperte = []
    bs_aperte = []
    Z_corrente = float("inf")

    while True:
        miglior_Z_step = Z_corrente
        centro_scelto = None
        ruolo_scelto = None
        centri_chiusi = [j for j in J if j not in bc_aperte and j not in bs_aperte]

        for j in centri_chiusi:
            # Test BC
            Z_con_bc = valutazione(bc_aperte + [j], bs_aperte)
            if Z_con_bc < miglior_Z_step:
                miglior_Z_step = Z_con_bc
                centro_scelto = j
                ruolo_scelto = "BC"

            # Test BS
```

```

        if len(bc_aperte) > 0:
            Z_con_bs = valutazione(bc_aperte, bs_aperte + [j])
            if Z_con_bs < miglior_Z_step:
                miglior_Z_step = Z_con_bs
                centro_scelto = j
                ruolo_scelto = "BS"

    if centro_scelto is None:
        break

    if ruolo_scelto == "BC":
        bc_aperte.append(centro_scelto)
    elif ruolo_scelto == "BS":
        bs_aperte.append(centro_scelto)

    Z_corrente = miglior_Z_step

    return Z_corrente, bc_aperte, bs_aperte

```

6.3 Drop Euristic

A differenza della Greedy costruttiva, questa euristica parte da una configurazione iniziale in cui tutti i nodi candidati sono aperti come Blood Center e nessuna Blood Station è ancora attiva. Ad ogni iterazione, l'algoritmo analizza uno alla volta i BC aperti e verifica se conviene modificarne lo stato. Per ogni centro candidato j vengono valutate due alternative: trasformare il BC in Blood Station, oppure chiuderlo completamente. Sceglie a ogni passo la modifica che produce il miglior valore della funzione obiettivo Z . Se la trasformazione in BS o la chiusura del centro riduce il costo complessivo, l'algoritmo aggiorna la rete; se invece nessuna modifica migliora la soluzione corrente, la procedura termina.

```

def Greedy_al_Contrario():
    bc_aperte = [j for j in J]
    bs_aperte = []
    Z_corrente = valutazione(bc_aperte, bs_aperte)

    while True:
        miglior_Z_step = Z_corrente
        centro_scelto = None
        ruolo_scelto = None

        for j in bc_aperte:
            bc_aperte_lessJ = [bc for bc in bc_aperte if bc != j]

            Z_con_bs = valutazione(bc_aperte_lessJ, bs_aperte + [j])
            if Z_con_bs < miglior_Z_step:
                miglior_Z_step = Z_con_bs
                centro_scelto = j
                ruolo_scelto = "BS"

```

```

Z_chiuso = valutazione(bc_aperte_lessJ, bs_aperte)
if Z_chiuso < miglior_Z_step:
    miglior_Z_step = Z_chiuso
    centro_scelto = j
    ruolo_scelto = "Closed"

if centro_scelto is None:
    break

bc_aperte.remove(centro_scelto)
if ruolo_scelto == "BS":
    bs_aperte.append(centro_scelto)

Z_corrente = miglior_Z_step

return Z_corrente, bc_aperte, bs_aperte

```

6.4 Confronto dei risultati tra euristiche e risolutore esatto

Per valutare l'efficacia delle euristiche implementate, abbiamo confrontato i risultati ottenuti con due approcci di tipo Greedy con quelli derivanti da una soluzione esatta calcolata tramite il solver Gurobi. Il problema è stato risolto per la regione Campania, considerando i parametri di progetto fissati come segue: target di autosufficienza regionale $D = 500000$, capacità massima di raccolta $C = 30000$, produttività minima obbligatoria per un BC $P_{\min} = 40000$, e capacità massima delle unità mobili $tn = 100000$.

I risultati ottenuti sono riassunti nella Tabella 1.

Metodo	Valore funzione obiettivo	Scostamento dall'ottimo	Gap %
Ottimo	3.261449×10^9	0	0%
Greedy	3.764217×10^9	5.02768×10^8	15.42%
Greedy al Contrario	3.310742×10^9	4.9293×10^7	1.51%

Table 1: Confronto dei valori della funzione obiettivo tra la soluzione ottima e le euristiche Greedy implementate.

Come evidenziato, la Greedy al Contrario si avvicina maggiormente alla soluzione ottima rispetto alla Greedy costruttiva. Questo risultato conferma l'efficacia dell'approccio Drop Heuristic nel migliorare progressivamente la configurazione della rete, partendo da una soluzione iniziale in cui tutti i centri candidati sono aperti come Blood Center. Al contrario, la Greedy costruttiva, sebbene rapida, soffre di mioopia locale e tende a tagliare alcune scelte ottimali già nelle prime iterazioni, producendo un gap maggiore rispetto alla soluzione ottima.

Questa analisi quantitativa permette di validare le euristiche implementate come strumenti efficaci per la generazione di soluzioni ammissibili e vicine all'ottimo, fornendo un punto di partenza per ulteriori strategie di miglioramento come la ricerca locale o la decomposizione del problema in sottoproblemi di tipo Simple Plant Location e Trasporto.

6.5 Limiti della Greedy

L'algoritmo Greedy è stato testato **variando diversi fattori**: il numero di centri candidati, la capacità delle strutture C , la disponibilità logistica tn e i requisiti di domanda e produttività. Quando il numero di centri candidati è basso o le risorse sono molto limitate, il problema diventa fortemente vincolato: in questi casi, lo spazio delle soluzioni ammissibili è ridotto e le euristiche tendono ad avvicinarsi maggiormente all'ottimo.

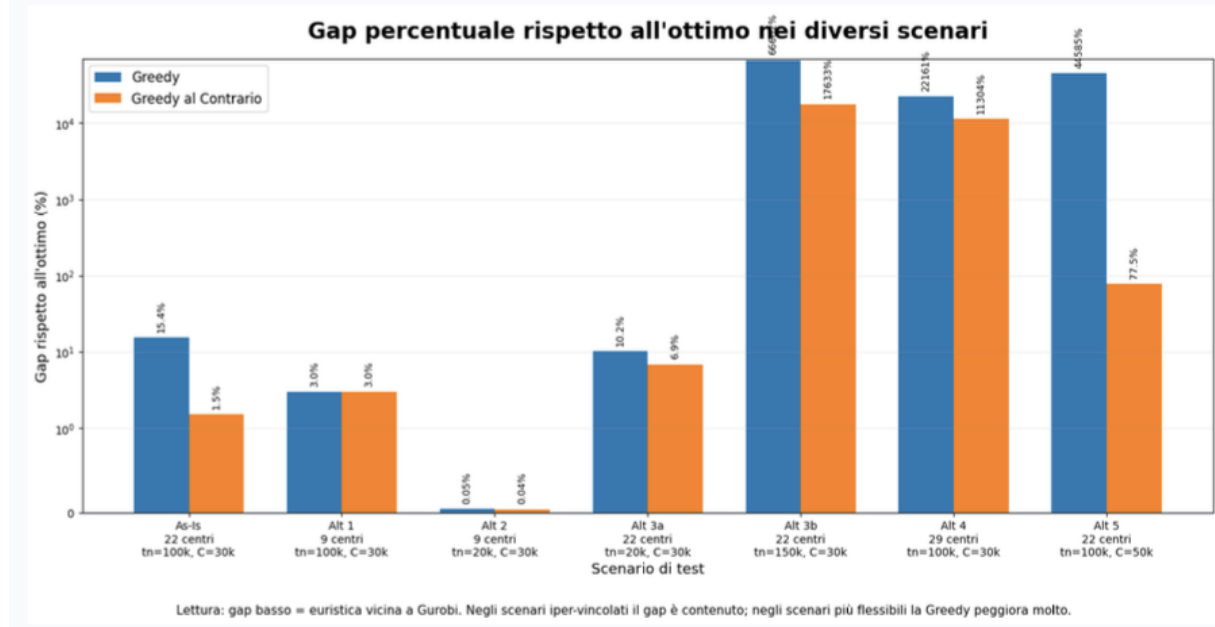


Figure 3: Gap percentuale rispetto all'ottimo

Al contrario, **all'aumentare del numero di centri candidati, lo spazio delle soluzioni cresce esponenzialmente**. L'algoritmo Greedy, essendo **miope**, ad ogni iterazione prende decisioni locali senza considerare l'impatto globale sulle scelte future, eliminando così alcune soluzioni potenzialmente migliori. Di conseguenza, i risultati ottenuti possono discostarsi significativamente dalla soluzione ottima. Lo stesso fenomeno si osserva anche fissando il numero di centri candidati ma variando i parametri di capacità, budget logistico o produttività: in funzione dei valori scelti, lo spazio delle soluzioni può aumentare o diminuire, comportando peggioramenti o miglioramenti dell'euristica. Per ridurre l'impatto di questi limiti, è possibile partire dalla soluzione ammissibile generata dall'euristica Greedy e applicare tecniche migliorative come *tabu search* o ricerca locale. In alternativa, si può considerare una scomposizione del problema in un sottoproblema di tipo *Simple Plant Location* e un problema di trasporto, permettendo di gestire in maniera più efficace lo spazio delle soluzioni e di avvicinarsi ulteriormente all'ottimo globale.

7 Conclusioni e Applicazioni

Il progetto mostra come un approccio combinato di MILP e euristiche consenta di:

- Riorganizzare la rete di centri trasfusionali mantenendo l'autosufficienza.
- Ridurre i costi logistici e rispettare vincoli di capacità.

- Adattarsi a scenari differenti variando parametri come raggio di copertura, capacità dei centri e disponibilità logistica.

Questo tipo di approccio è applicabile ad altre regioni o reti logistiche di servizi critici, **combinando modelli di ottimizzazione rigorosi** con **euristiche scalabili** per ottenere soluzioni pratiche.

References

- [1] A. Diglio, A. Mancuso, A. Masone, C. Piccolo, C. Sterle, *A MILP Formulation for the Reorganization of the Blood Supply Chain in Italian Regions*, 2021.